# Operating System Structure

Dr.H.Summia Parveen,AP/CSE

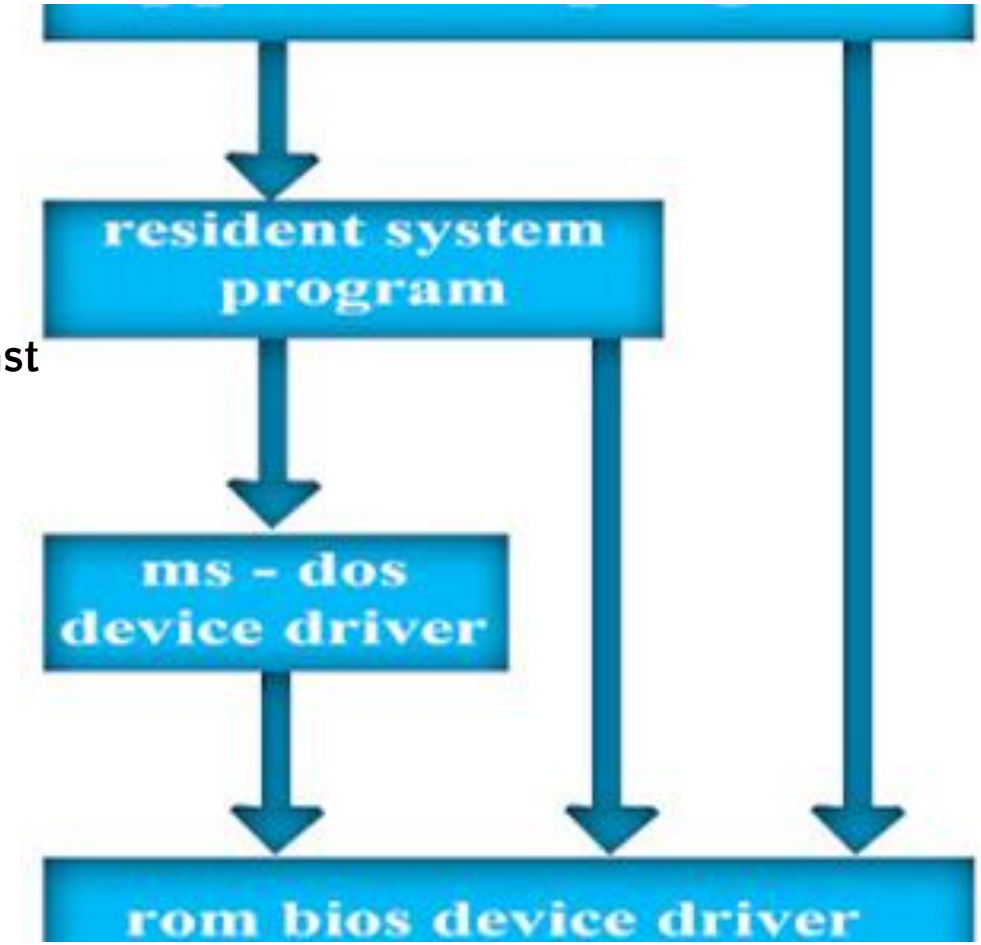
# Operating System Structure

Operating system is partitioned into separate subsystems, each with carefully defined tasks, input, output, and function. This arrangement of and relations between the subsystems or parts of complex system is called operating system structure."

# Simple Structure

**MS-DOS – written to provide the most functionality in the least space**

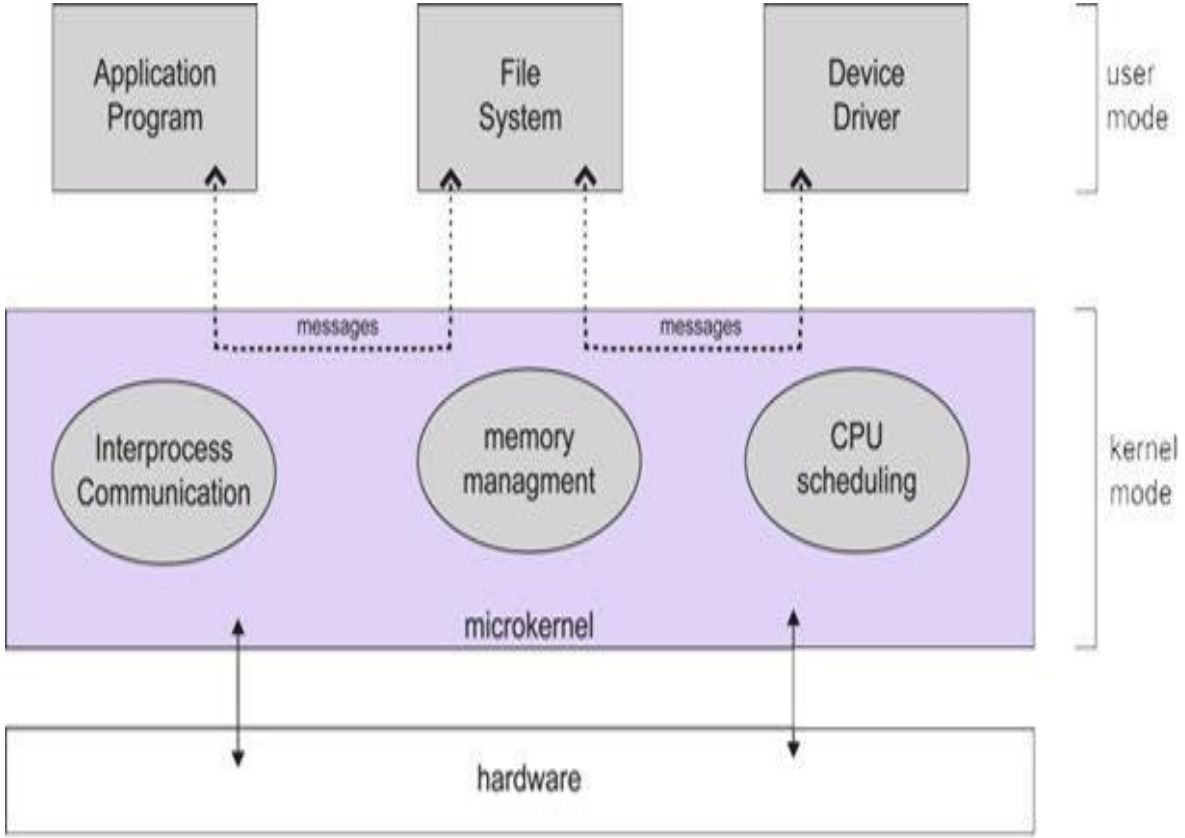
- Not divided into modules
- Although MS-DOS has some structure, its interfaces and levels of functionality are not well separated

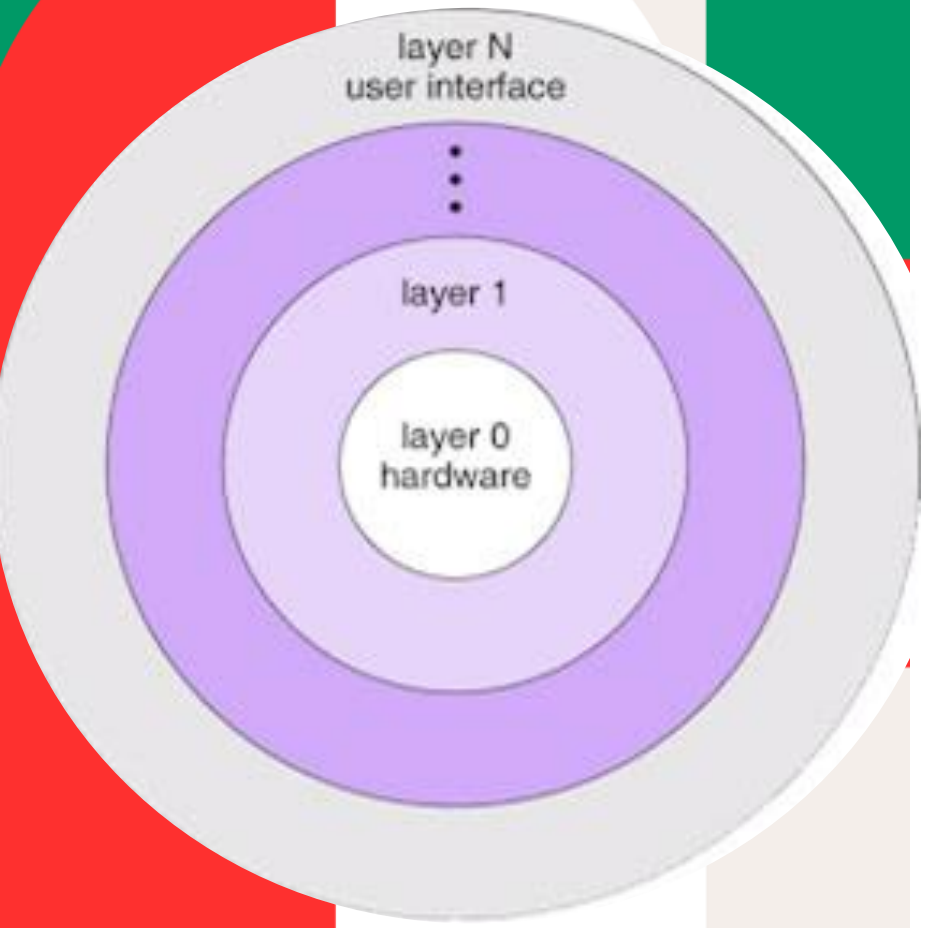


# Micro kernels

**Mac OSX**

- It removes all non essential components from the kernel, and implements them as system- and user level programs. There by making the kernel as small and efficient as possible.
- The main function of the micro kernel is to provide a communication facility between the client program and the various services that are also running in user space.
- Communication is provided by message passing
- The benefits of the micro kernel approach is ease of extending the operating system.
- The micro kernel also provides more security and reliability. If a service fails, the rest of the operating system remains untouched



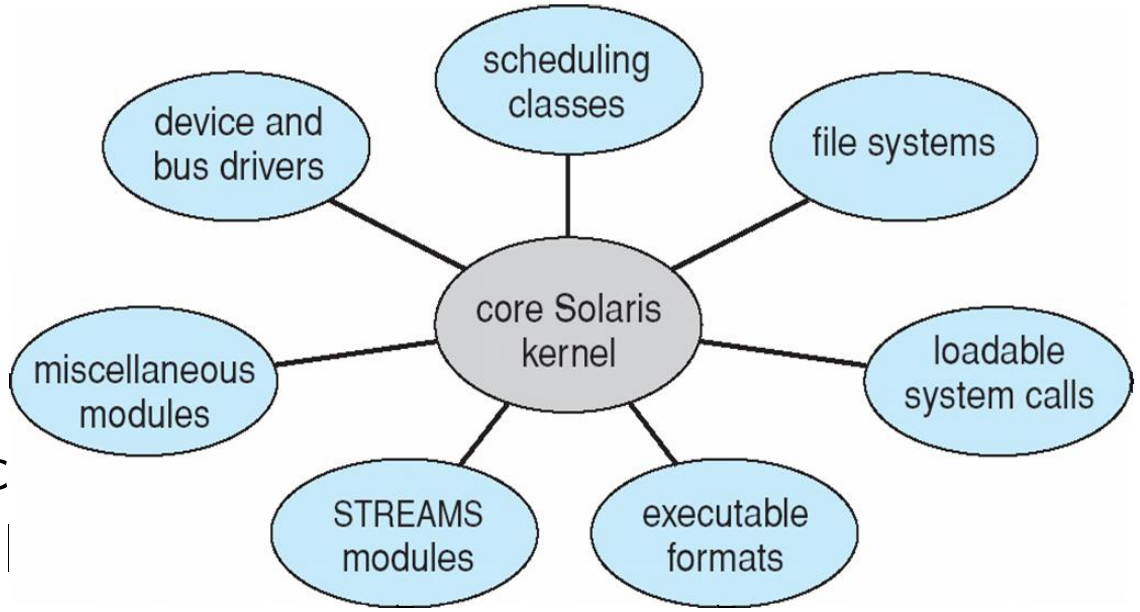



# Layered Approach

- The operating system is divided into a number of layers (levels), each built on top of lower layers. The bottom layer (layer 0), is the hardware; the highest (layer N) is the user interface.
- With modularity, layers are selected such that each uses functions (operations) and services of only lower-level layers

# Modules

- Many modern operating systems implement **loadable kernel modules**
  - Uses object-oriented approach
  - Each core component is separate
  - Each talks to the others over known interfac
  - Each is loadable as needed within the kernel
- Overall, similar to layers but with more flexible
  - Linux, Solaris, etc



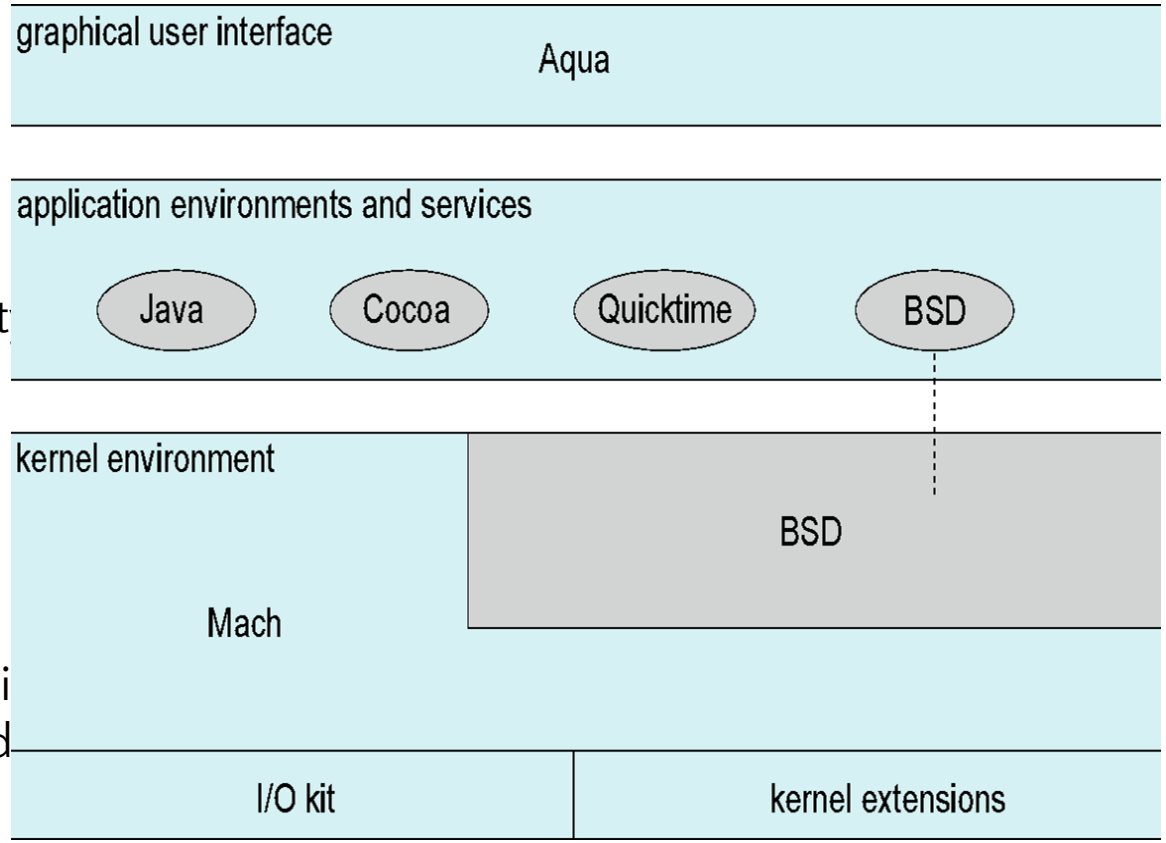
# Hybrid Systems

**Most modern operating systems are actually not one pure model**

- Hybrid combines multiple approaches to address performance, security, usability needs
- Linux and Solaris kernels in kernel address space, so monolithic, plus modular for dynamic loading of functionalit
- Windows mostly monolithic, plus microkernel for different subsystem ***personalities***

**Apple Mac OS X hybrid, layered, Aqua UI plus Cocoa programming environment**

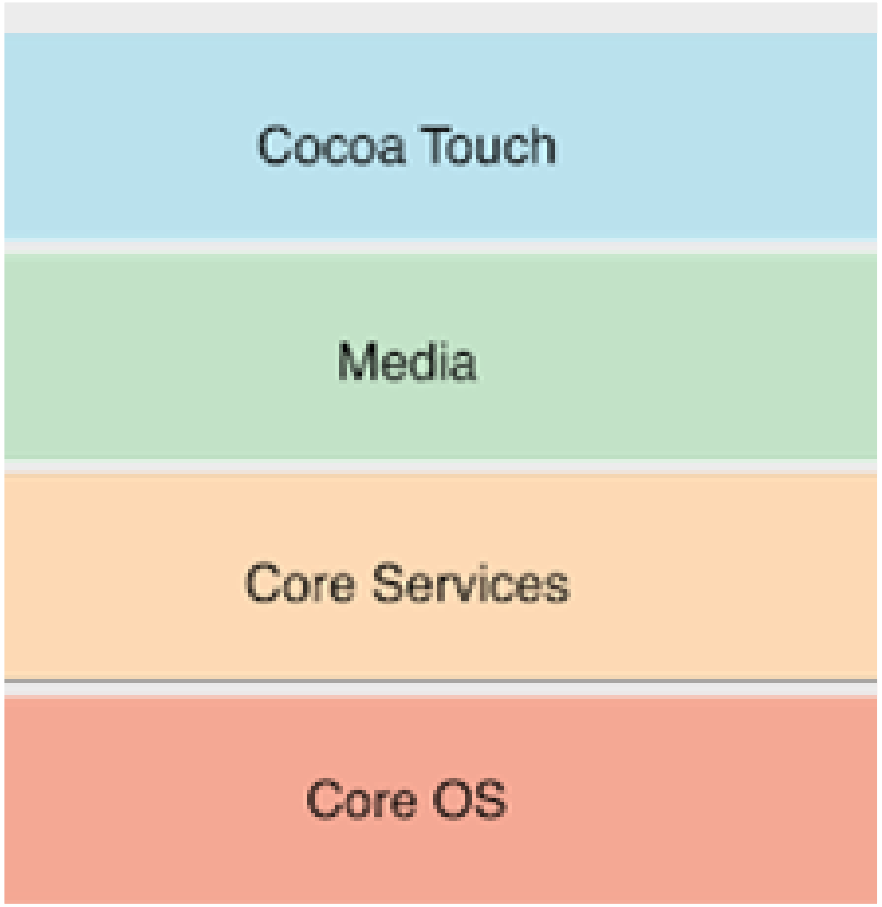
- Below is kernel consisting of Mach microkernel and BSD Uni parts, plus I/O kit and dynamically loadable modules (called kernel extensions)

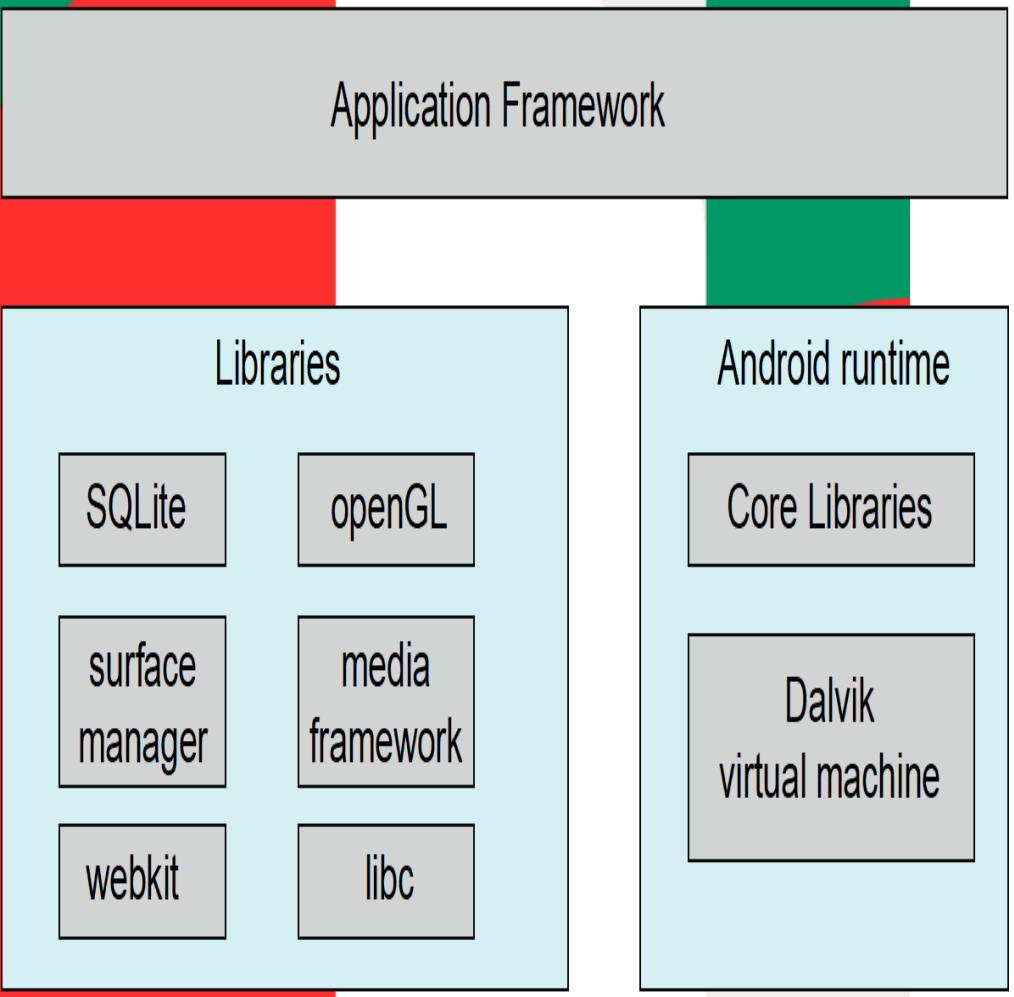


# iOS

**Apple mobile OS for *iPhone*, *iPad***

- Structured on Mac OS X, added functionality
- Does not run OS X applications natively
    - Also runs on different CPU architecture (ARM vs. Intel)
- **Cocoa Touch** Objective-C API for developing apps
- **Media services** layer for graphics, audio, video
- **Core services** provides cloud computing, databases
- Core operating system, based on Mac OS X kernel






# Android

**Developed by Open Handset Alliance (mostly Google)**

- Open Source

**Similar stack to IOS**

**Based on Linux kernel but modified**

- Provides process, memory, device-driver management
- Adds power management

**Runtime environment includes core set of libraries and Dalvik virtual machine**

- Apps developed in Java plus Android API
    - Java class files compiled to Java bytecode then translated to executable than runs in Dalvik VM

**Libraries include frameworks for web browser (webkit), database (SQLite), multimedia, smaller libc**